

Análisis y diseño con patrones

PR1: Principios de diseño, patrones de análisis y arquitectónicos

Índice

Compromiso.....	2
Pregunta 1 (20 %).....	3
Pregunta 1.1 (10 %).....	4
Pregunta 1.2 (10 %).....	6
Pregunta 2 (30 %).....	7
Pregunta 2.1 (5 %).....	7
Pregunta 2.2 (10 %).....	8
Pregunta 2.3 (15 %).....	9
Pregunta 3 (30 %).....	11
Pregunta 3.1. (5 %).....	11
Pregunta 3.2 (10 %).....	12
Pregunta 3.3 (15 %).....	12
Pregunta 4 (20 %).....	15
Pregunta 4.1. (5 %).....	15
Pregunta 4.2. (15 %).....	15

Compromiso

Certifico que he realizado la PRA1 de forma completamente individual y únicamente con la ayuda que el profesorado de esta asignatura considera oportuna, según las indicaciones explicadas en el documento “Originalidad en la evaluación” disponible en el aula. Entiendo que el trabajo no original y/o el empleo de IA generativa implicará que la actividad entregada no será corregida y que automáticamente se le asignará una calificación de 0 %.

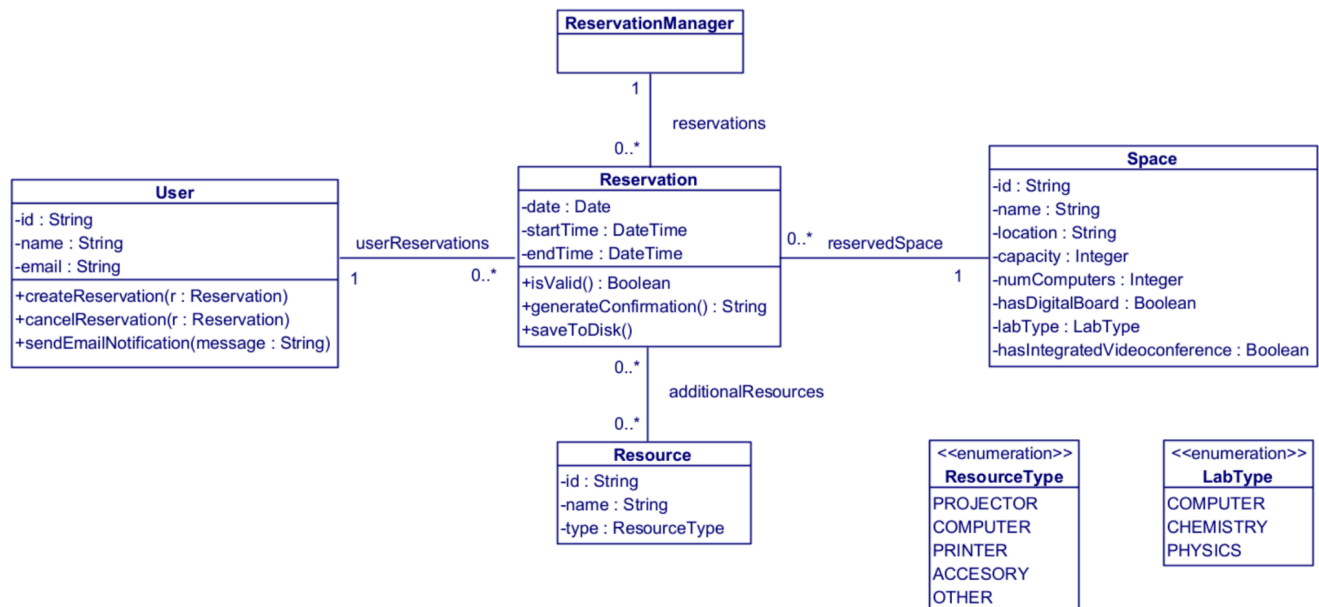
Pregunta 1 (20 %)

Una universidad quiere desarrollar un sistema para gestionar la reserva de espacios y recursos del campus. El sistema debe permitir a los usuarios solicitar reservas de distintos tipos de espacios, como aulas, laboratorios o salas de reuniones, así como de ciertos recursos adicionales, como proyectores, cámaras o kits de videoconferencia que se pueden llevar al espacio reservado.

Para los espacios debemos almacenar su identificador, nombre, ubicación y capacidad. Los laboratorios deben incluir también el tipo de laboratorio y el número de ordenadores; las aulas deben indicar si tienen pizarra digital; y las salas de reuniones deben indicar si disponen de videoconferencia integrada.

Cada reserva debe indicar al menos el usuario que la solicita, el espacio principal reservado, el momento previsto de uso y, opcionalmente, otros recursos adicionales asociados.

Se ha propuesto el siguiente diseño inicial para el sistema:



- La clase *Reservation* almacena los datos del usuario, del espacio reservado, de los recursos adicionales, de la fecha y de las horas de inicio y fin.
- La misma clase *Reservation* calcula si la reserva es válida (*isValid()*), comprueba conflictos con otras reservas y genera el mensaje de confirmación (*generateConfirmation()*); y guarda la reserva en disco (*saveToDisk()*).
- La clase *ReservationManager* se encarga de mantener toda la lista de reservas activas, pero delega en la clase *Reservation* la lógica de validación, comprobación de conflictos, generación de mensajes y persistencia.
- La clase *Space* incluye atributos comunes y específicos de todos los tipos de espacios, de modo que una misma clase representa aulas, laboratorios y salas de reuniones.

- La clase *User* mantiene una lista de sus reservas y contiene operaciones para crear y cancelar reservas, así como enviar avisos por correo electrónico.

Restricciones de seguridad:

- Los identificadores de espacios y recursos deben ser únicos.
- Las reservas no pueden solaparse en el tiempo para el mismo espacio.
- Los recursos adicionales a una reserva no pueden solaparse con otras reservas.

Además, la universidad quiere que el sistema sea fácil de ampliar, ya que en el futuro podrían incorporarse nuevos tipos de espacios, nuevos recursos y distintos mecanismos de notificación o persistencia.

Pregunta 1.1 (10 %)

Analiza críticamente este diseño inicial e indica qué **principios de diseño** del temario no se están respetando. Justifica la respuesta.

El diseño propuesto no respeta los siguientes principios de diseño:

- **Alta cohesión**

Sabemos que una clase tiene alta cohesión cuando sus métodos y atributos están relacionados y enfocados en un único propósito. En este caso, la clase **Reservation** tiene una cohesión **muy baja**, ya que mezcla la representación de los datos de la reserva con la lógica de negocio (*isValid*), la lógica de presentación (*generateConfirmation*) y la lógica de persistencia (*saveToDisk*).

Por otra parte, la clase **User** mezcla la información del usuario con la gestión de reservas (*createReservation*, *cancelReservation*) y con funciones de comunicación externa (*sendEmailNotification*).

- **Principio abierto-cerrado**

Este principio dicta que las entidades de software deben estar abiertas a la extensión, pero cerradas a la modificación. Como podemos ver en la clase **Space**, su estructura provoca que si la universidad quiere añadir un nuevo tipo de espacio en el futuro, se tendría que abrir y modificar la clase *Space*. Lo correcto sería usar polimorfismo con una clase base (o interfaz) *Space* y clases derivadas para cada espacio concreto (*Lab*, *MeetingRoom*, *Classroom*...).

También apreciamos que al tener métodos como *sendEmailNotification* o *saveToDisk*, si mañana se quiere cambiar el medio de notificación (por ejemplo, SMS) o la unidad en la que guardar la información, como en una base de datos relacional, habría que modificar el código fuente de *User* y *Reservation*.

- **Inversión de dependencias**

Este principio establece que los módulos de alto nivel no deben depender de los módulos de

bajo nivel, sino que ambos deben depender de abstracciones. En este caso, comprobamos que las clases del núcleo del dominio, **Reservation** y **User**, están acopladas a implementaciones tecnológicas específicas: el sistema de archivos (*saveToDisk()*) y un servidor de correo electrónico (*sendEmailNotification()*). Deberían depender de interfaces abstractas, como *IReservationRepository* o *INotificationService*, que se inyecten desde fuera.

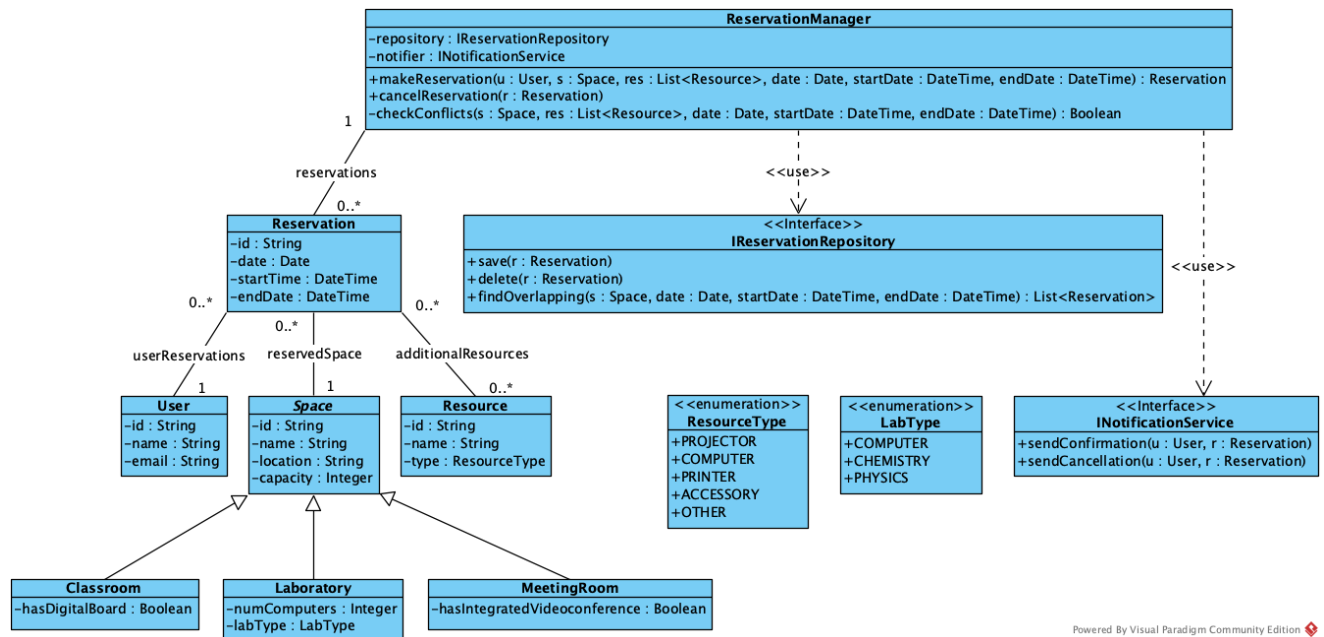
- **Bajo acoplamiento**

El acoplamiento mide el grado de dependencia entre distintos módulos. Debido a la violación del principio de **inversión de dependencias** mencionada arriba, el sistema tiene un **alto acoplamiento** con su infraestructura. Si la librería o el sistema operativo que gestiona la escritura en disco cambia, la clase *Reservation* se verá afectada.

Además, delegar toda la lógica de validación de conflictos desde **ReservationManager** a la propia *Reservation* genera un acoplamiento incoherente a la lógica de negocio, ya que una reserva individual no debería tener la responsabilidad de conocer el estado de todas las demás reservas para saber si se solapa. Esa lógica de coordinación debería pertenecer a un servicio o a la propia clase *ReservationManager*.

Pregunta 1.2 (10 %)

Propón un rediseño del modelo conceptual y plásmalo mediante un **diagrama de clases UML**.



Para solucionar los problemas de diseño identificados en la pregunta anterior y cumplir con los principios SOLID, creamos la clase abstracta **Space** con los atributos comunes. De ella heredan **Classroom**, **Laboratory** y **MeetingRoom**, cada una con sus atributos específicos.

Por otra parte, hemos extraído las operaciones de infraestructura a interfaces puras: **IReservationRepository**: se encarga de la persistencia y **INotificationService** se encarga de las notificaciones.

De forma más general, ahora las entidades son puras, ya que solo almacenan su estado y sus relaciones. Ya no envían correos ni se guardan a sí mismas en disco. Para ello, hemos creado la clase **ReservationManager** para que realice todas las operaciones, como recibir las peticiones, comprobar los conflictos llamando al repositorio (*checkConflicts()*), crear el objeto **Reservation** usando **IReservationRepository** y enviar notificaciones con **INotificationService**.

Pregunta 2 (30 %)

La universidad detecta que realizar una reserva no debe comprobar únicamente solapamientos con otras reservas, sino también validar que esté fuera de periodos de mantenimiento. Estos periodos de mantenimiento se asignan de forma global a toda la universidad para todas las reservas y recursos, por ejemplo, en los periodos intercuatrimestrales.

Por ello, se quiere revisar el modelo temporal del sistema para representar adecuadamente la franja horaria asociada a una reserva y reutilizar ese mismo concepto en otras situaciones.

Podéis asumir la existencia de una clase *MaintenancePeriod* que almacenará una instancia de un periodo de mantenimiento.

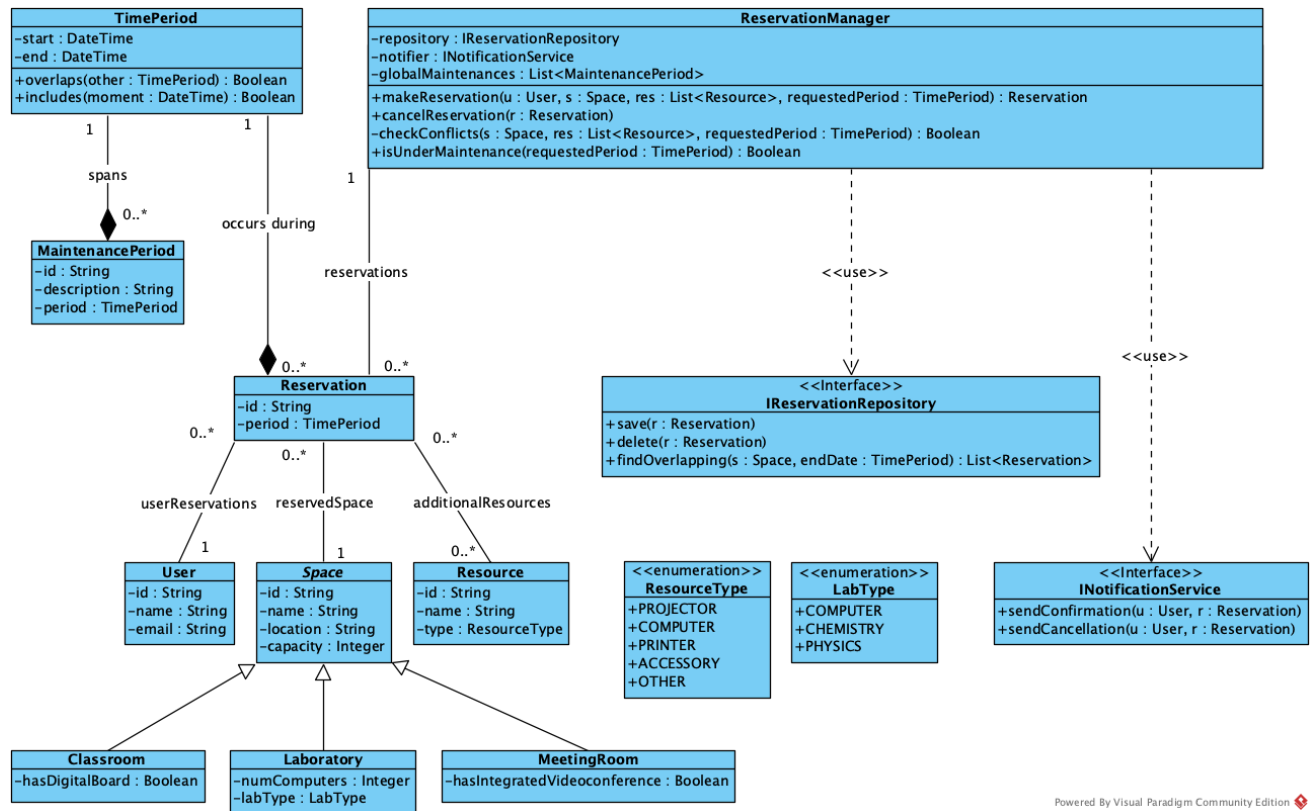
Pregunta 2.1 (5 %)

Justifica qué **patrón de análisis** se debe aplicar para agregar esta nueva funcionalidad.

Actualmente, el sistema controla el tiempo de forma rudimentaria, con los atributos *date*, *startTime* y *endTime* sueltos dentro de la clase *Reservation*. Aplicar el patrón **Range** implica extraer estos atributos para encapsularlos en una nueva clase dedicada con tal de poder reutilizarla en varias clases y aumentar la cohesión del sistema al centralizar el cálculo del tiempo en esta nueva clase.

Pregunta 2.2 (10 %)

Muestra el **diagrama de clases** con las operaciones necesarias para aplicar este diseño. Para este diseño **no podéis utilizar clases de la librería de Java**, aunque exista una clase aplicable, esperamos que hagáis vuestro propio diseño.



Como se solicita que sea un diseño propio sin utilizar clases de la librería de Java, como *java.time.Interval*, creamos la clase **TimePeriod** (arriba a la izquierda) desde cero con la lógica necesaria para controlar las comparaciones temporales. Como podemos ver, en lugar de recibir los tres parámetros sueltos (*Date d*, *DateTime startDate* y *DateTime endDate*), los sustituimos por un parámetro de tipo *TimePeriod*.

Pregunta 2.3 (15 %)

Muestra el **pseudocódigo** de las clases que participan en el diseño, empezando desde la implementación del método que valida una reserva, comprobando las restricciones con otras reservas y con los periodos de mantenimiento.

```
public class ReservationManager {
    private IReservationRepository repository;
    private INotificationService notifier;
    private List<MaintenancePeriod> globalMaintenances;

    public Reservation makeReservation(User user, Space space, List<Resource>
resources, TimePeriod requestedPeriod) {
        if (isUnderMaintenance(requestedPeriod)) {
            throw new IllegalStateException("Error: El sistema está en
mantenimiento en esas fechas.");
        }

        if (checkConflicts(space, resources, requestedPeriod)) {
            throw new IllegalStateException("Error: El espacio o los recursos
ya están ocupados en ese periodo.");
        }

        Reservation newReservation = new Reservation(user, space, resources,
requestedPeriod);
        repository.save(newReservation);
        notifier.sendConfirmation(user, newReservation);

        return newReservation;
    }

    public void cancelReservation(Reservation reservation) {
        repository.delete(reservation);
        notifier.sendCancellation(reservation.getRequestedBy(), reservation);
    }

    private boolean checkConflicts(Space space, List<Resource> resources,
TimePeriod requestedPeriod) {
        List<Reservation> overlappingReservations =
repository.findOverlapping(space, requestedPeriod
        return !overlappingReservations.isEmpty();
    }

    private boolean isUnderMaintenance(TimePeriod requestedPeriod) {
        for (MaintenancePeriod maintenance : globalMaintenances) {
            if (maintenance.getPeriod().overlaps(requestedPeriod)) {
```

```

        return true;
    }
}
return false;
}
}

public class TimePeriod {
    private DateTime start;
    private DateTime end;

    public TimePeriod(DateTime start, DateTime end) {
        if (end.isBefore(start)) {
            throw new IllegalArgumentException("La fecha de fin no puede ser
anterior a la de inicio.");
        }
        this.start = start;
        this.end = end;
    }

    public boolean overlaps(TimePeriod other) {
        return this.start.isBefore(other.end) && this.end.isAfter(other.start);
    }
}

public class MaintenancePeriod {
    private String id;
    private String description;
    private TimePeriod period;
}

public class Reservation {
    private String id;
    private User requestedBy;
    private Space reservedSpace;
    private List<Resource> additionalResources;
    private TimePeriod period;
}

public interface IReservationRepository {
    void save(Reservation r);
    void delete(Reservation r);
    List<Reservation> findOverlapping(Space s, TimePeriod period);
}

```

Pregunta 3 (30 %)

El sistema de reservas actual solo permite reservar recursos individuales, como un proyector o una cámara. Sin embargo, la universidad quiere ampliar el sistema para que también se puedan reservar conjuntos de recursos. Por ejemplo, un equipo de proyección puede incluir un proyector, un portátil y un cable VGA, o un kit para docencia híbrida puede incluir una cámara, un micrófono y un sistema de audio;

El sistema debe permitir trabajar con todos estos elementos de manera uniforme, de forma que sea posible: consultarlos, añadirlos a una reserva, y obtener información agregada, como el número total de recursos incluidos.

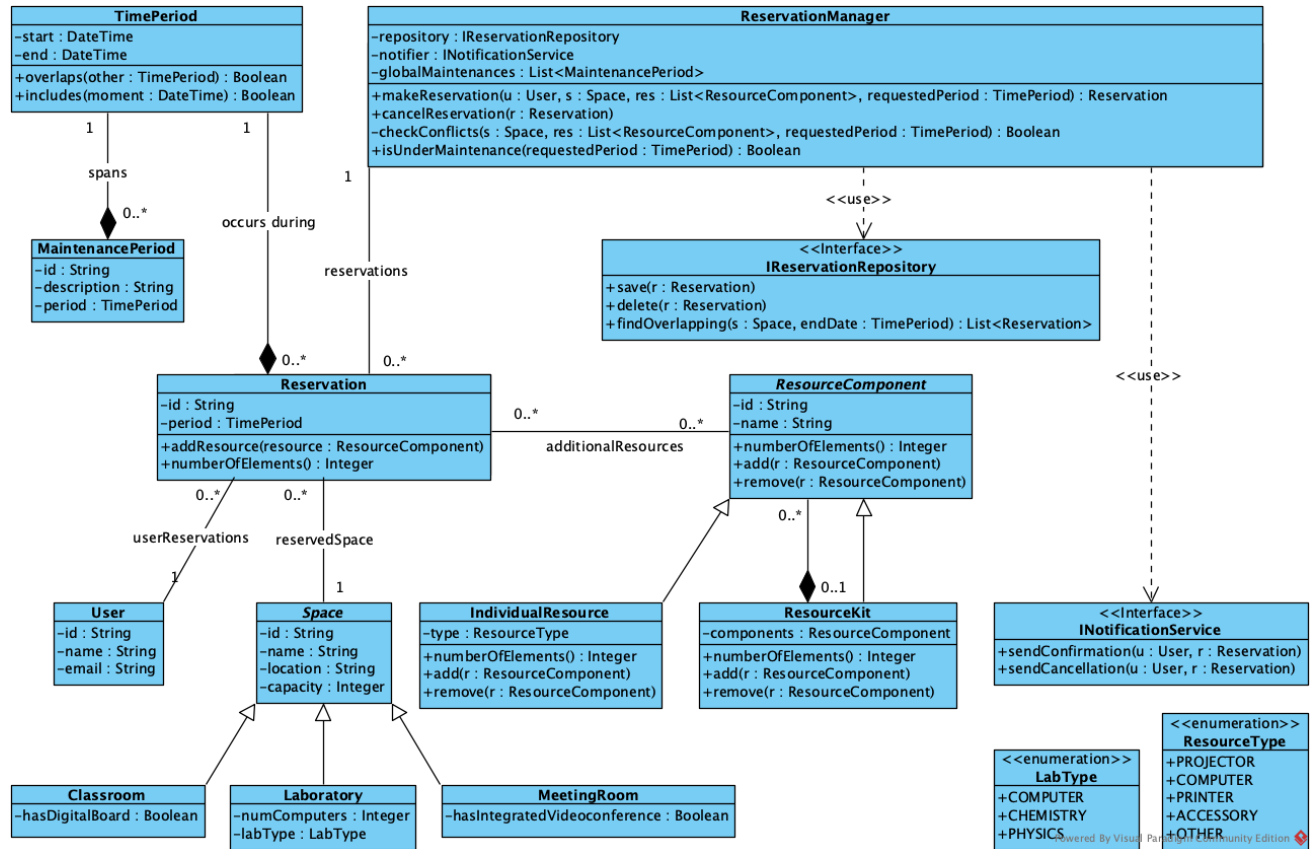
Pregunta 3.1. (5 %)

Justifica qué **patrón de análisis** se adapta mejor a esta nueva necesidad

El enunciado pide que el sistema pueda consultar y añadir a una reserva tanto recursos sueltos como conjuntos "de manera uniforme". Con el patrón **Composite**, extraemos una interfaz común (por ejemplo, *ResourceComponent*). Tanto el recurso individual (*IndividualResource*) como el kit (*ResourceKit*) implementarán esta interfaz. Así, la clase *Reservation* no necesita saber si le asignamos un solo cable o un kit completo de docencia híbrida. Simplemente tratará con la abstracción *ResourceComponent*.

Pregunta 3.2 (10 %)

Muestra el **diagrama de clases** con las operaciones necesarias para implementar este diseño, así como sus restricciones de integridad.



Pregunta 3.3 (15 %)

Muestra el **pseudocódigo completo** de las clases que participan en la operación *Reservation::numberOfElements()* que calcula el número total de recursos incluidos en la reserva.

```
public abstract class ResourceComponent {
    protected String id;
    protected String name;
    protected ResourceType type;

    public abstract int numberOfElements();

    public void add(ResourceComponent component) {
        throw new UnsupportedOperationException("No se pueden añadir elementos a este componente.");
    }

    public void remove(ResourceComponent component) {
        throw new UnsupportedOperationException("No se pueden eliminar elementos de este componente.");
    }
}

public class IndividualResource extends ResourceComponent {
    private ResourceType type;

    @Override
    public int numberOfElements() {
        return 1;
    }
}

public class ResourceKit extends ResourceComponent {
    private List<ResourceComponent> components;

    @Override
    public int numberOfElements() {
        int total = 0;
        for (ResourceComponent component : components) {
            total += component.numberOfElements();
        }
        return total;
    }

    @Override
```

```

    public void add(ResourceComponent component) {
        components.add(component);
    }

    @Override
    public void remove(ResourceComponent component) {
        components.remove(component);
    }
}

public class Reservation {
    private String id;
    private List<ResourceComponent> additionalResources;

    public void addResource(ResourceComponent resource) {
        this.additionalResources.add(resource);
    }

    public int numberOfElements() {
        int totalElements = 0;
        for (ResourceComponent resource : additionalResources) {
            totalElements += resource.numberOfElements();
        }

        return totalElements;
    }
}

```

Pregunta 4 (20 %)

La universidad prevé que, en futuras versiones del sistema, la gestión de reservas deba poder trabajar con distintos servicios externos, por ejemplo:

- Diferentes mecanismos de notificación al usuario, como correo electrónico o mensajería interna;
- o distintos sistemas de persistencia de datos;

La dirección técnica ha decidido que las clases responsables de gestionar las reservas **no deben crear directamente** estos servicios concretos, ya que se quiere poder sustituir una implementación por otra sin modificar la lógica principal del sistema.

Por ejemplo, la misma lógica de creación o cancelación de reservas debería poder funcionar con un notificador por correo electrónico o con un notificador de mensajería interna. O la base de datos podría ser PostgreSQL o MySQL, sin que eso afecte a la clase que gestiona las reservas.

Pregunta 4.1. (5 %)

Justifica qué **patrón arquitectónico** se adecúa mejor a esta necesidad

La **inyección de dependencias** encaja con la descripción del problema, ya que elimina la creación directa de objetos al prohibir que *ReservationManager* cree un nuevo servicio de notificaciones o un repositorio. En su lugar, el gestor simplemente pide que le entreguen esos servicios ya creados, normalmente a través de su constructor.

Como el gestor de reservas solo conoce las interfaces, como *INotificationService*, podemos inyectarle cualquier clase que cumpla con ese contrato. Si al arrancar el programa le inyectamos el notificador por correo, enviará correos. Por otra parte, si le inyectamos uno de mensajería interna, enviará mensajes. Todo esto ocurre sin tocar ni una sola línea de código de la lógica de reservas.

Pregunta 4.2. (15 %)

Muestra el **diagrama de clases** con las operaciones necesarias para implementar este diseño en el que se vea claramente cómo una clase encargada de gestionar reservas recibe desde fuera las dependencias que necesita, sin instanciar implementaciones concretas.

